Classificando 102 espécies de flores

Modalidade: classificação de imagens. Cada imagem recebe uma única classe, entre as 102 espécies do Flowers102.

Entrada: imagem RGB de 224 x 224 pixels, já normalizada.
Saída: um vetor com 102 valores, e a classe prevista é o maior deles.

Nesta sprint o foco é a arquitetura da CNN. Partimos de uma rede pequena e mudamos um fator por vez, sempre comparando no Validation. O Test só foi usado uma vez, no modelo escolhido.

Grupo

Gabriel Fidalgo - RM563213

Pedro Lima - RM565461

Gustavo Maia - RM562240

Gustavo Rossi - RM566075

Mesmo dataset e mesma divisão da Sprint 3

Conjunto	Imagens	Observação
Train	1.020	10 por classe
Validation	1.020	10 por classe
Test	6.149	de 20 a 238 por classe

A divisão é a oficial do Flowers102 (a mesma que o torchvision entrega). Salvamos os caminhos em splits/*.csv e conferimos que bate com os números da Sprint 3 e que nenhum arquivo aparece em dois conjuntos. Pelo conteúdo, o dataset original tem 2 pares de imagens idênticas (train/test e val/test). Como a divisão não pode mudar, só registramos isso.



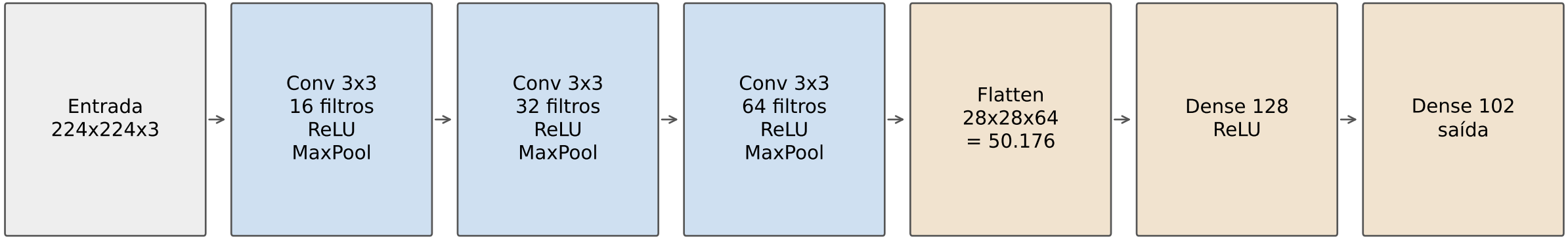
exemplos do Train (6 classes diferentes)

Pré-processamento

Resize para 256 x 256 e recorte de 224 x 224, depois normalização com média e desvio da ImageNet. Val e Test usam o recorte central. No Train o recorte é aleatório, com flip horizontal e vertical e rotação de até 30 graus. A Sprint 3 descrevia esse fluxo no relatório, mas o notebook dela só redimensionava direto para 224, então seguimos o relatório.

Não usamos class_weight: o Train é balanceado, então os pesos seriam todos iguais a 1.

Baseline: 3 blocos Conv-ReLU-MaxPool e Flatten



6.459.398 parâmetros no total. Quase tudo está na primeira Dense: 6.422.656 parâmetros (99.4%), porque o Flatten entrega 50.176 valores. Os três blocos de convolução juntos têm só 23.584.

Protocolo igual para todos os experimentos

Adam com lr 5e-4, batch 32, até 50 épocas, parada antecipada pela acurácia do Validation (paciência 10), 3 seeds (42, 43 e 44). Usamos lr 5e-4 porque com 1e-3 alguns modelos travaram no acaso logo na primeira época (testamos isso só em 3 configurações, seed 42).

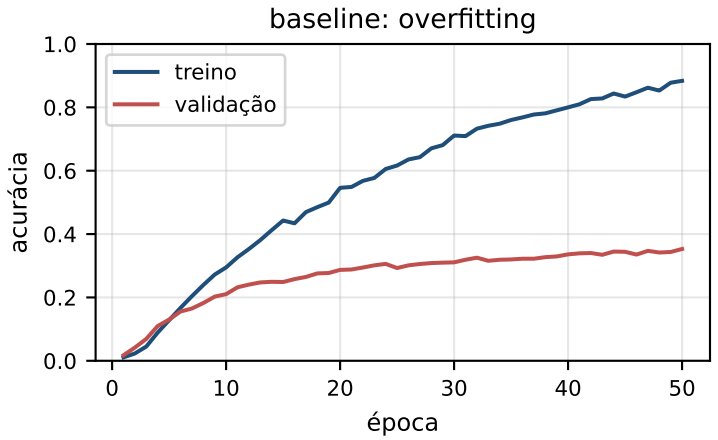
O que mudamos em cada experimento

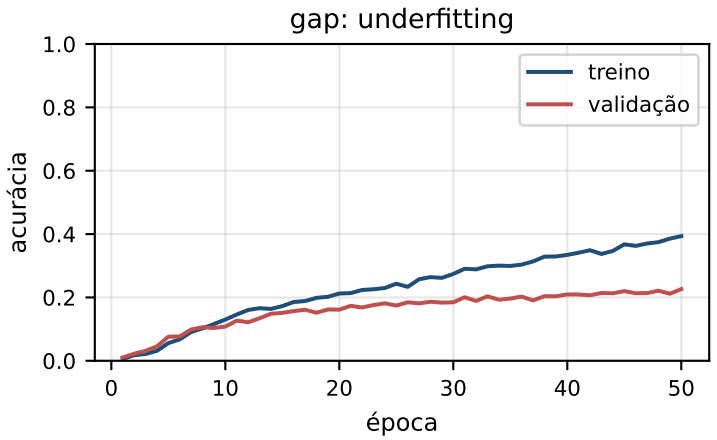
Experimento	Mudança (uma por vez em relação ao baseline)	Por que testamos
baseline	Arquitetura inicial	Ponto de partida
prof_2	2 blocos (16-32)	Ver se o terceiro bloco era necessário
prof_4	4 blocos (16-32-64-128)	Mais campo receptivo e Flatten menor
largura	Filtros 32-64-128	Mais padrões por camada
kernel_5	Kernel 5x5	Olhar uma região maior de cada vez
stride	Conv com stride 2, sem MaxPool	Ver se o MaxPool joga informação fora
gap	GAP no lugar do Flatten	Cortar parâmetros da cabeça
denso_256	Dense 256	Mais capacidade ou só mais parâmetros?
gap_bn	GAP + BatchNorm (muda só o BN em relação ao gap)	O BN foi mal com Flatten, testamos com a cabeça pequena
batchnorm	BatchNorm depois de cada conv	Estabilizar o treino

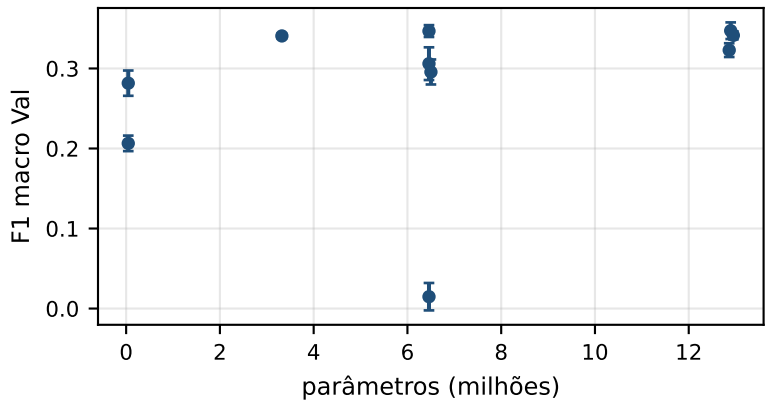
O gap_bn foi adicionado depois de ver que o batchnorm não treinou. Ele é comparado com o gap, e não com o baseline.

Validation: quase tudo empata ou piora

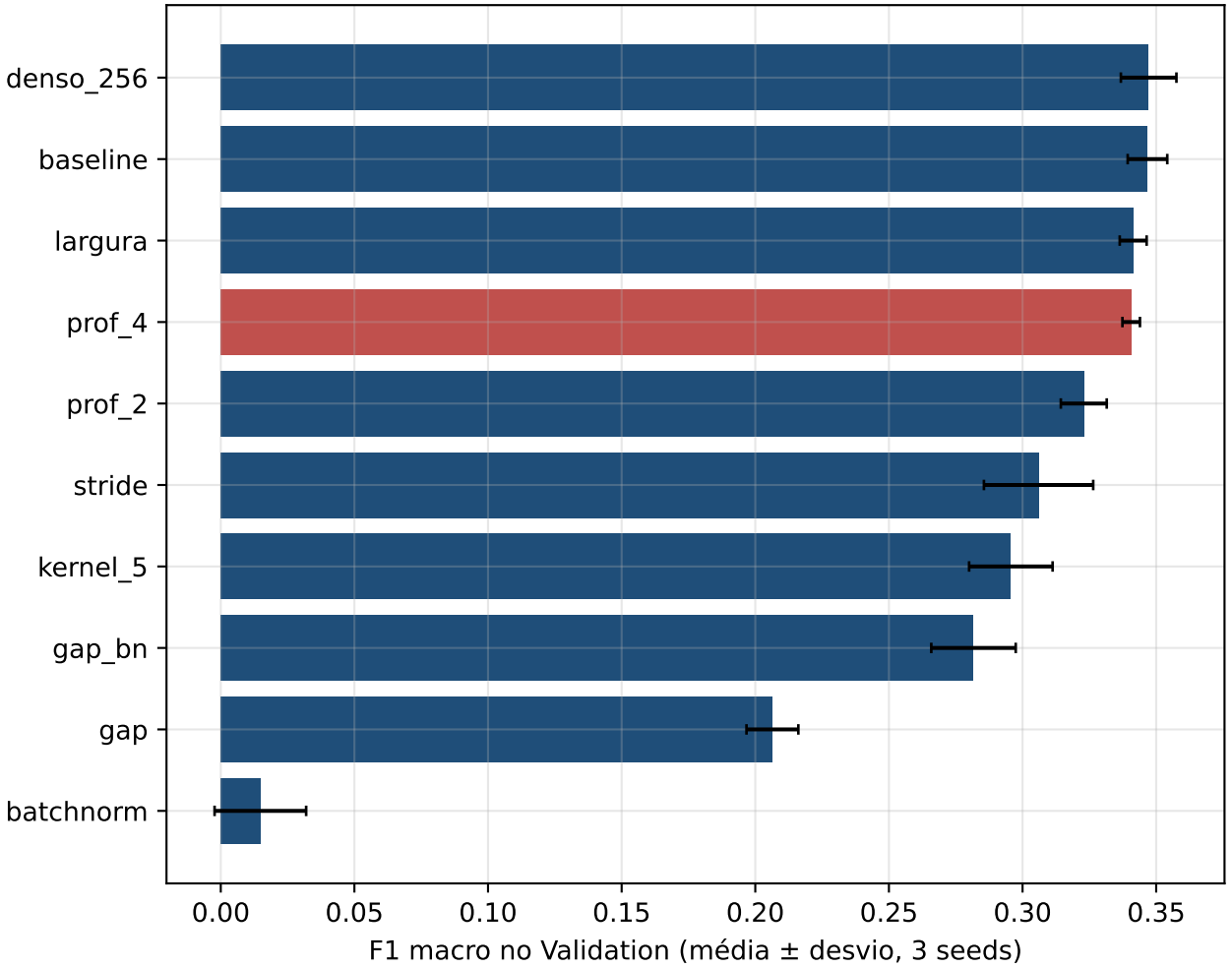
Experimento	Mudança principal	Parâmetros	F1 macro Validation (média ± desvio)	Observação
baseline	Arquitetura inicial	6.459.398	0,347 ± 0,007	Ponto de partida, overfitting
prof_2	2 blocos (16-32)	12.863.430	0,323 ± 0,009	Piorou, e a cabeça dobrou de tamanho
prof_4	4 blocos (16-32-64-128)	3.321.990	0,341 ± 0,003	Empata com metade dos parâmetros
largura	Filtros 32-64-128	12.951.590	0,341 ± 0,005	Empata, mas treina bem mais devagar
kernel_5	Kernel 5x5	6.501.126	0,296 ± 0,016	Piorou
stride	Conv com stride 2, sem MaxPool	6.459.398	0,306 ± 0,020	Piorou um pouco
gap	GAP no lugar do Flatten	45.062	0,206 ± 0,010	Underfitting
denso_256	Dense 256	12.895.110	0,347 ± 0,010	Empata, só mais parâmetros
gap_bn	GAP + BatchNorm (BN sobre o gap)	45.286	0,282 ± 0,016	Melhor que o gap, mas ainda abaixo
batchnorm	BatchNorm depois de cada conv	6.459.622	0,015 ± 0,017	Não aprendeu com Flatten







Escolhemos o prof_4: 4 blocos, metade dos parâmetros



Regra definida antes de rodar: pegamos o maior F1 macro médio e, entre os modelos a menos de 1 desvio dele, ficamos com o de menos parâmetros.

Ficaram empatados: baseline, prof_4, largura, denso_256. O prof_4 tem F1 0,341 e 3.321.990 parâmetros, contra 6.459.398 do baseline (F1 0,347).

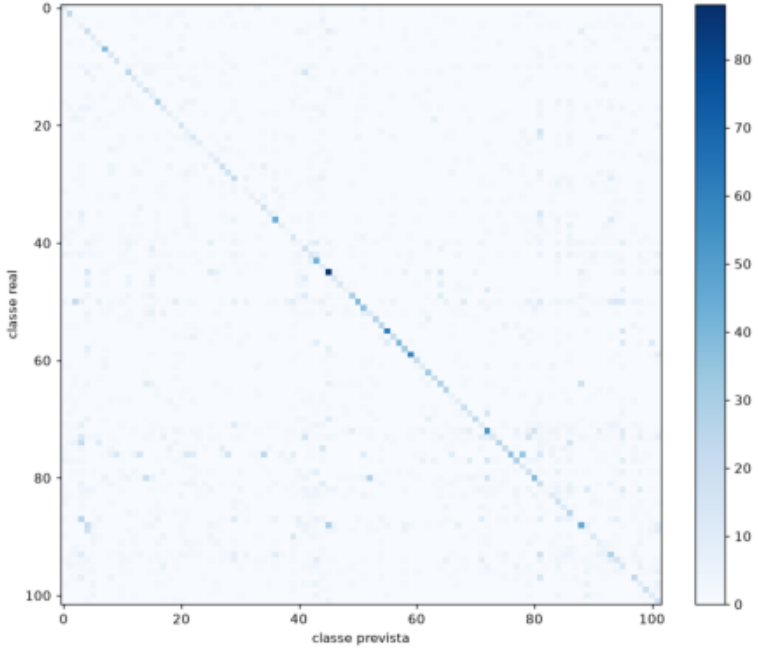
Por que faz sentido: mais um bloco de convolução diminui o mapa antes do Flatten (de 28x28 para 14x14), e a Dense 128 passa a ter metade dos pesos. O Flatten gigante era o que mais pesava no baseline. Ganhamos a mesma qualidade com um modelo menor.

Não dá para dizer que ele é melhor que o baseline, só que não é pior. Escolhemos pela simplicidade.

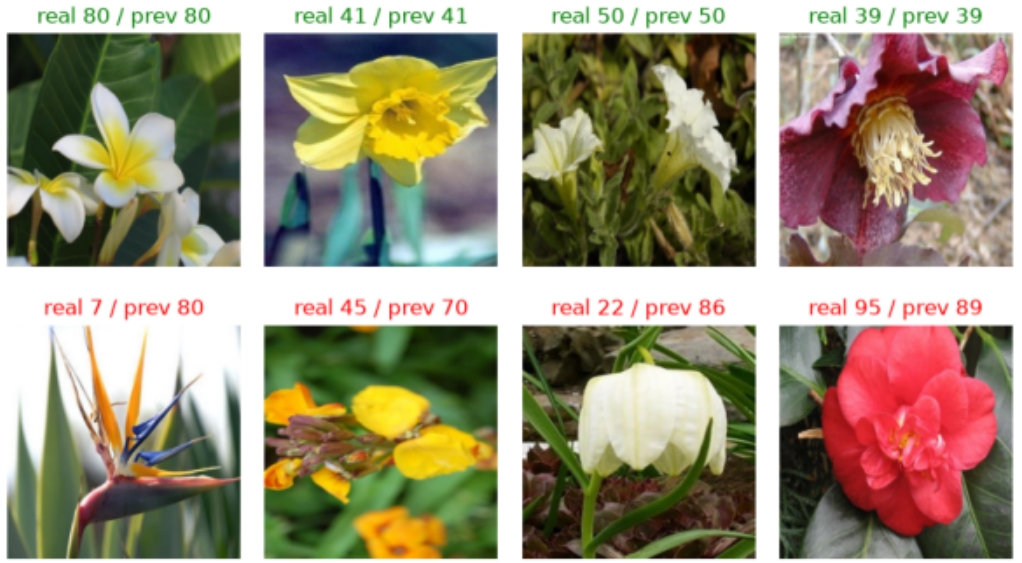
Test: acurácia de 0,300 no prof_4

Métrica (6.149 imagens)	seed 42	média ± desvio das 3 seeds
Acurácia	0,291	0,300 ± 0,009
Precision macro	0,306	0,311 ± 0,004
Recall macro	0,324	0,330 ± 0,004
F1 macro	0,287	0,293 ± 0,004

Matriz de confusão (102 x 102)



Acertos (em cima) e erros (embaixo), seed 42



No Validation o F1 era 0,341, e no Test ficou em 0,293. A queda é esperada, já que escolhemos épocas e modelo olhando o Validation.

Pares mais confundidos (real/previsto): 76/78 (35 imagens), 88/45 (29) e 80/52 (26). 3 classes tiveram F1 zero. Usamos o ID da classe, como na Sprint 3.

GAP foi o que mais pesou, e o overfitting é o problema

O que mais pesou

Entre as mudanças que treinaram direito, trocar o Flatten pelo GAP teve o maior efeito: o F1 caiu de 0,347 para 0,206 e o modelo ficou em underfitting, com só 45.062 parâmetros. Perder a posição espacial das features, ou ficar pequeno demais, custou caro. O BatchNorm com Flatten caiu mais ainda (F1 0,015), mas foi falha de treino e não dá para concluir nada sobre BN. Kernel 5x5, stride no lugar do MaxPool e 2 blocos pioraram um pouco. Largura, Dense 256 e 4 blocos ficaram dentro da variação entre seeds.

Limitações

Os modelos com Flatten fazem overfitting forte (acurácia de treino entre 0,672 e 0,871 contra 0,312 a 0,360 no Validation) e a melhor época caiu perto do limite de 50, então talvez ainda desse para treinar mais. Foram só 3 seeds por configuração, e com 10 imagens por classe a rede aprende pouco do zero. No Test, 3 classes ficaram com F1 zero, e isso não parece ter relação com o tamanho da classe.

Próximos passos

Testar transfer learning com uma rede pré-treinada na ImageNet, que é o caminho natural com tão poucas imagens por classe. Também queremos olhar dropout, weight decay e augmentation mais forte para atacar o overfitting, e repetir o BatchNorm com um lr menor.